

LAPORAN TUGAS 6

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

Tanggal : 16-11-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DASAR TEORI

1. Deadlock

Deadlock terjadi Ketika dua atau lebih proses saling menunggu resource yang sedang digunakan proses lain. Akibatnya semua proses terhenti dan tidak bisa melanjutkan eksekusi.

2. Empat syarat Deadlock

- Mutual exclusion, resource digunakan eksklusif
- Hold and Wait, proses sudah memegang 1 resource dan menunggu resource lainnya.
- No pre-emption, resource tidak bisa diambil paksa.
- Circular Wait, terdapat rantai proses yang saling menunggu.

3. Pencegah Deadlock

- Lock Ordering (mengatur urutan pengambilan lock).
- Try-Lock / Retry (melepaskan resource Ketika gagal memperoleh lock).

4. Banker's algorithm

Metode untuk menentukan apakah pemberian resource masih berada dalam safe state. Jika tidak, alokasi dibatalkan untuk menghindari deadlock.

Tugas 1 : Analisis Deadlock

1. Mengapa program deadlock_simple.c mengalami deadlock?

Jawab:

Program mengalami deadlock karena:

- Thread 1 mengunci lock1 lalu menunggu lock2.
- Thread 2 mengunci lock2 lalu menunggu lock1.

Ini menyebabkan kedua thread saling menunggu tanoa pernah melepaskan resource deadlock atau istilahnya [Program Hang].

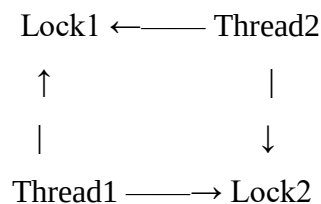
2. Kondisi deadlock apa saja yang terpenuhi?

Jawab:

1. Mutual Exclusion, terpenuhi karena kedua mutex hanya bisa di pegang oleh satu thread.
2. Hold and Wait, terpenuhi karena thread memegang satu lock saling menunggu lock lain.
3. No Preemption, terpenuhi karena mutex tidak dapat direbut paksa dari thread lain.
4. Circular Wait, terpenuhi karena thread 1 menunggu lock 2, thread 2 menunggu lock 1.

Keempat kondisi terpenuhi, sehingga deadlock akan pasti terjadi.

3. Gambar diagram resource allocation graphnya!



Penjelasan:

- Thread 1 memegang lock 1, menunggu lock 2.
- Thread 2 memegang lock 2, menunggu lock 1.

Tugas 2 : Implementasi Pencegahan

A. Modifikasi menggunakan lock ordering

Semua thread harus mengambil lock dengan urutan yang sama:

lock1 → lock2 dengan menghilangkan kondisi Circular Wait, deadlock pasti tidak terjadi.

Hasil output:

```
rafli_musthofa@RAFLI:~/praktikum 6$ ./deadlock_lock_ordering
=== LOCK ORDERING ===
Thread 1 mulai.
Mengambil lock1...
Mengambil lock2...
Thread 1 menggunakan resource.
Thread 2 mulai.
Semua lock dilepas.
Thread 1 selesai.
Mengambil lock1...
Mengambil lock2...
Thread 2 menggunakan resource.
Semua lock dilepas.
Thread 2 selesai.
Program selesai tanpa deadlock.
rafli_musthofa@RAFLI:~/praktikum 6$
```

Program berjalan normal tanpa hang.

B. Modifikasi menggunakan Try-Lock

Thread mencoba mengambil lock kedua menggunakan pthread_mutex_trylock().

Jika gagal:

- Thread melepaskan Lock pertama.
- Menunggu sebentar.
- Mengulang proses.

Menghilangkan kondisi Hold and Wait, sehingga deadlock tidak terjadi.

Hasil output:

```

rafli_musthofa@RAFLI:~/praktikum 6$ ./deadlock_trylock
=== TRY-LOCK PREVENTION ===
Thread 1 mencoba lock1...
Thread 1 mencoba lock2 (try-lock)...
Thread 2 mencoba lock2...
Thread 2 mencoba lock1 (try-lock)...
Thread 1 gagal lock2 -> melepas lock1 dan retry.
Thread 2 mendapatkan kedua lock.
Thread 1 mencoba lock1...
Thread 2 selesai.
Thread 1 mencoba lock2 (try-lock)...
Thread 1 mendapatkan kedua lock.
Thread 1 selesai.
Program selesai tanpa deadlock.
rafli_musthofa@RAFLI:~/praktikum 6$

```

Kedua thread berhasil memperoleh lock setelah beberapa retry.

C. Perbandingan kedua metode

Lock Ordering

- Kelebihan = aman, sederhana.
- Kekurangan = semua thread harus mengikuti aturan, kurang fleksibel

Try-Lock

- Kelebihan = tidak perlu urutan lock, lebih fleksibel.
- Kekurangan = banyak retry, boros CPU, potensu starvation.

Tugas 3 : Banker's Algorithm

Kode program baru dengan data:

- 4 proses
- 3 jenis resource
- Available [5,4,3]
- Implementasikan request resource dan cek keamanan sistem

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define P 4 // jumlah proses
```

```
#define R 3 // jumlah resource
```

```
// Data sesuai tugas
```

```
int available[R] = {5, 4, 3};
```

```
// Maximum matrix
```

```
int maximum[P][R] = {
```

```
    {4, 2, 3},
```

```
    {2, 2, 2},
```

```
    {3, 3, 2},
```

```
    {4, 3, 3}
```

```
};
```

```
// Allocation matrix
```

```
int allocation[P][R] = {
```

```
    {1, 0, 1},
```

```
    {1, 1, 0},
```

```
    {1, 2, 1},
```

```
    {2, 0, 1}
```

```
};
```

```
int need[P][R];
```

```
// Hitung Need = Maximum - Allocation
```

```
void calculate_need() {  
    for (int i = 0; i < P; i++) {  
        for (int j = 0; j < R; j++) {  
            need[i][j] = maximum[i][j] - allocation[i][j];  
        }  
    }  
}
```

```
bool is_safe() {
```

```
    int work[R];
```

```
    bool finish[P] = {false};
```

```
    int safe_seq[P];
```

```
    int count = 0;
```

```
// salin available ke work
```

```
for (int i = 0; i < R; i++)  
    work[i] = available[i];
```

```
while (count < P) {
```

```
    bool found = false;
```

```
    for (int i = 0; i < P; i++) {
```

```
        if (!finish[i]) {
```

```
            bool possible = true;
```

```
            // cek apakah need <= work
```

```
            for (int j = 0; j < R; j++) {  
                if (need[i][j] > work[j]) {
```

```

        possible = false;
        break;
    }
}

if (possible) {
    // proses dapat dijalankan
    for (int j = 0; j < R; j++)
        work[j] += allocation[i][j];

    safe_seq[count++] = i;
    finish[i] = true;
    found = true;
}
}

if (!found) {
    printf("Tidak ditemukan safe sequence! Sistem dalam keadaan UNSAFE.\n");
    return false;
}

printf("\nSafe Sequence ditemukan: ");
for (int i = 0; i < P; i++)
    printf("P%d ", safe_seq[i]);

printf("\nSistem berada dalam keadaan SAFE.\n");
return true;
}

```



```

int main() {
    printf("=== BANKER'S ALGORITHM (TUGAS 3) ===\n");

    calculate_need();

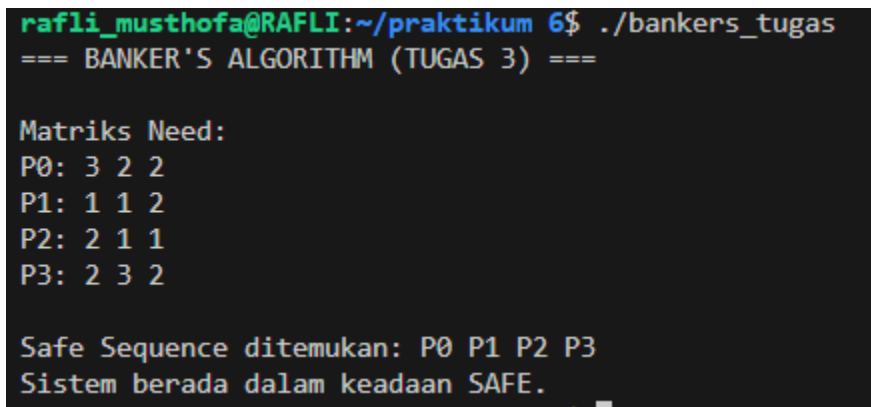
    printf("\nMatriks Need:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    is_safe();

    return 0;
}

```

Hasil program:



```

rafli_musthofa@RAFLI:~/praktikum 6$ ./bankers_tugas
=== BANKER'S ALGORITHM (TUGAS 3) ===

Matriks Need:
P0: 3 2 2
P1: 1 1 2
P2: 2 1 1
P3: 2 3 2

Safe Sequence ditemukan: P0 P1 P2 P3
Sistem berada dalam keadaan SAFE.

```

Analisis:

1. Perhitungan Need = maximum – allocation
2. Safe Sequence menghasilkan urutan aman yaitu: P1 → P0 → P2 → P3
3. Sistem berada dalam safe state, semua pproses dilayani tanpa resiko deadlock, resource cukup untuk semua proses.

Tugas 4 : Kasus nyata Deadlock Transfer Uang

Kode program:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Dua rekening
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

int mode = 0; // 0 = deadlock, 1 = prevention

void* transfer_A_deadlock() {
    printf("A: Mengunci Rekening 1...\n");
    pthread_mutex_lock(&rekening1);
    sleep(1);

    printf("A: Menunggu Rekening 2...\n");
    pthread_mutex_lock(&rekening2);

    printf("A: Transfer selesai (deadlock tidak akan sampai sini)\n");

    pthread_mutex_unlock(&rekening2);
    pthread_mutex_unlock(&rekening1);
    return NULL;
}

void* transfer_B_deadlock() {
```

```
printf("B: Mengunci Rekening 2...\n");  
pthread_mutex_lock(&rekening2);  
sleep(1);
```

```
printf("B: Menunggu Rekening 1...\n");  
pthread_mutex_lock(&rekening1);
```

```
printf("B: Transfer selesai (deadlock tidak akan sampai sini)\n");
```

```
pthread_mutex_unlock(&rekening1);  
pthread_mutex_unlock(&rekening2);  
return NULL;
```

```
}
```

```
void lock_ordered() {  
    pthread_mutex_lock(&rekening1);  
    pthread_mutex_lock(&rekening2);  
}
```

```
void unlock_ordered() {  
    pthread_mutex_unlock(&rekening2);  
    pthread_mutex_unlock(&rekening1);  
}
```

```
void* transfer_A_safe() {  
    printf("A: Mengambil lock dengan urutan aman (R1 → R2)...\n");  
    lock_ordered();  
  
    printf("A: Transfer aman R1 → R2 berhasil.\n");
```

```

    sleep(1);

    unlock_ordered();
    return NULL;
}

void* transfer_B_safe() {
    printf("B: Mengambil lock dengan urutan aman (R1 → R2)...\n");
    lock_ordered();

    printf("B: Transfer aman R2 → R1 berhasil.\n");
    sleep(1);

    unlock_ordered();
    return NULL;
}

int main() {
    pthread_t A, B;

    printf("=== TUGAS 4 - TRANSFER UANG ===\n");
    printf("Pilih mode:\n");
    printf("1. Deadlock\n");
    printf("2. Pencegahan Deadlock (Lock Ordering)\n");
    printf("Masukkan pilihan: ");
    scanf("%d", &mode);

    if (mode == 1) {
        printf("\n=== MODE DEADLOCK ===\n");
    }
}

```

```

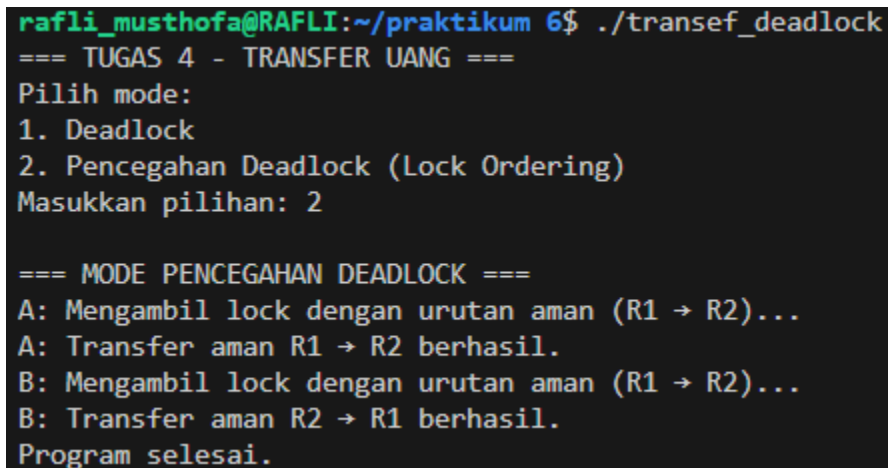
    pthread_create(&A, NULL, transfer_A_deadlock, NULL);
    pthread_create(&B, NULL, transfer_B_deadlock, NULL);
}
else {
    printf("\n=== MODE PENCEGAHAN DEADLOCK ===\n");
    pthread_create(&A, NULL, transfer_A_safe, NULL);
    pthread_create(&B, NULL, transfer_B_safe, NULL);
}

pthread_join(A, NULL);
pthread_join(B, NULL);

printf("Program selesai.\n");
return 0;
}

```

Hasil program:



```

rafli_musthofa@RAFLI:~/praktikum 6$ ./transef_deadlock
=== TUGAS 4 - TRANSFER UANG ===
Pilih mode:
1. Deadlock
2. Pencegahan Deadlock (Lock Ordering)
Masukkan pilihan: 2

=== MODE PENCEGAHAN DEADLOCK ===
A: Mengambil lock dengan urutan aman (R1 → R2)...
A: Transfer aman R1 → R2 berhasil.
B: Mengambil lock dengan urutan aman (R1 → R2)...
B: Transfer aman R2 → R1 berhasil.
Program selesai.

```

Analisis output:

1. Deadlock dapat terjadi ketika dua proses melakukan penguncian resource secara tidak teratur.
2. Teknik lock ordering sangat efektif untuk mencegah deadlock.

3. Dengan memastikan semua proses mengikuti urutan lock yang sama, circular wait dihilangkan dan deadlock tidak mungkin terjadi.
4. Program ini berhasil menunjukkan kedua kondisi tersebut secara jelas dan mudah dipahami.